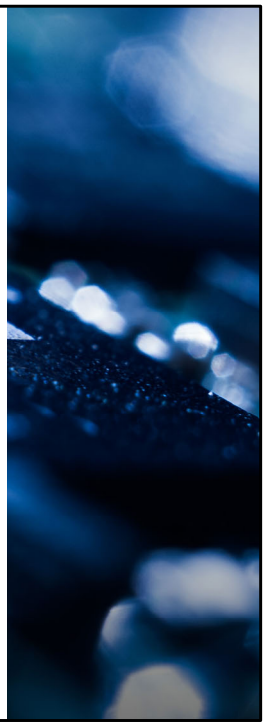
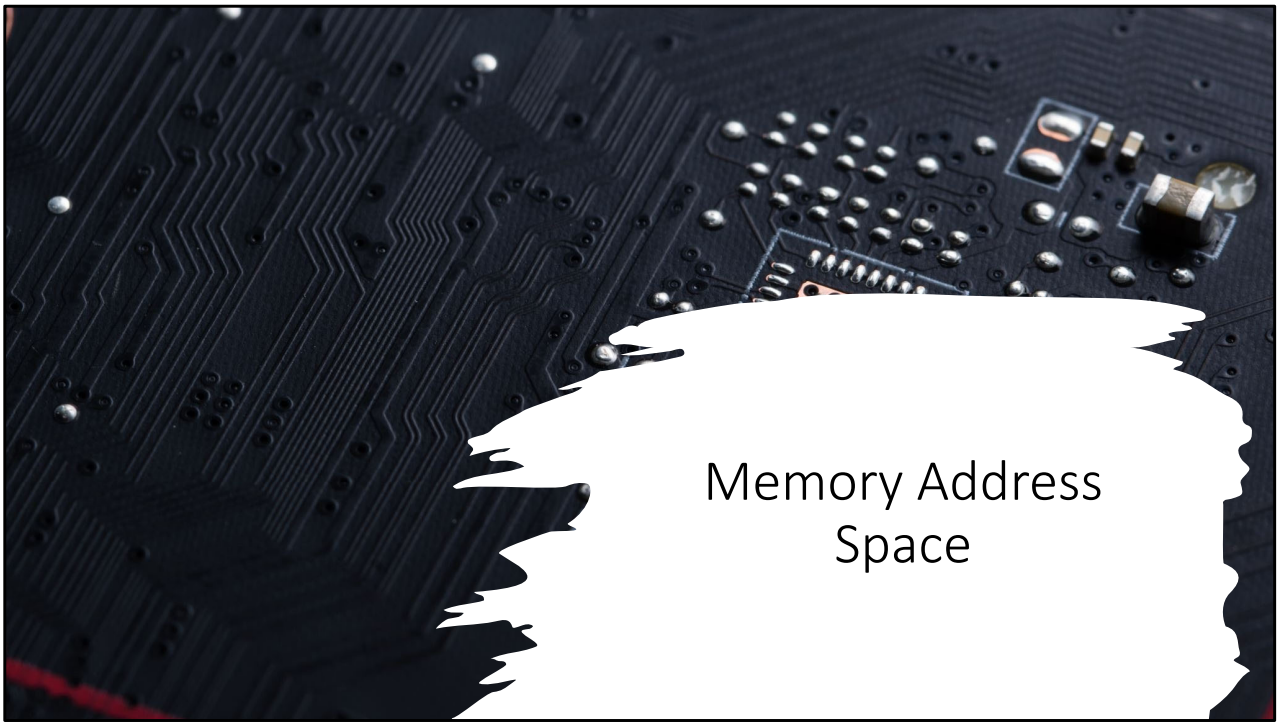
SC1006
Computer Organisation and Architecture
Virtual Memory

Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg



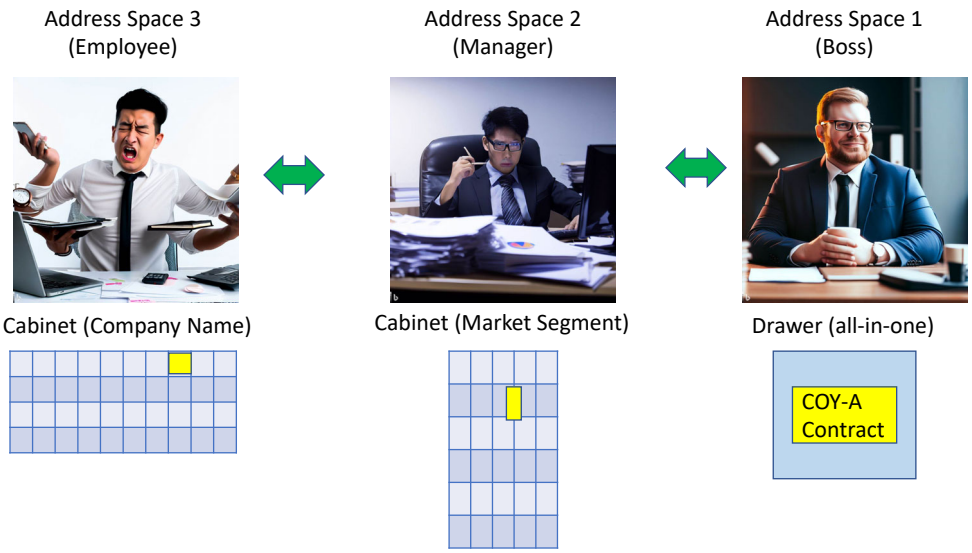
- This chapter is on Cache Memory Management. This is a module in the processor that is able to improve the overall system performance significantly based on the principle of locality exhibited by most application programs. More details in this video.
-



Memory Address Space

- Before we start on the Virtual Memory Management Topic, I would like to use a few slides to introduce the concept of memory address space in computing.

Memory Address Space (Analogy)



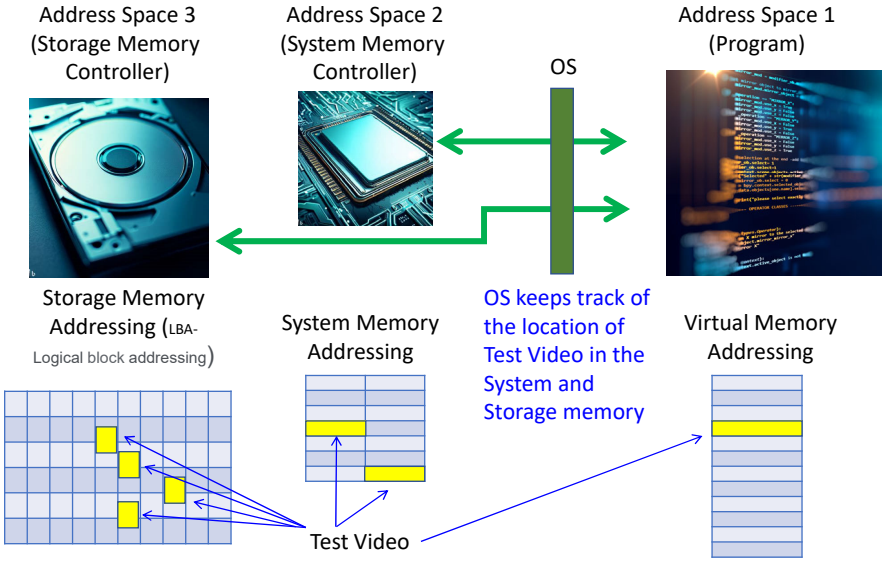
Let's start with an analogy.

Memory space are how a sub-system or module keep its information.

Take for example, in a company,
the big boss will keep the most important documents in his drawer.
His manager will probably have a larger cabinet consisting of the key contracts and information specific to the market segments they supervise.
The subordinates would likely have their own cabinet which they kept documents corresponding to the specific customers they call on.

The three parties do not know how each other store their information. But the company operation will still run smoothly, as long as, when the boss needs a specific piece of information, his managers or the subordinates under the managers can provide him the required data.
As long as the boss is happy, everything is ok.

Memory Address Space (Computing)



Coming back to computing.

Different entities within the computer stores information differently.
Each will have its own memory space and own addressing scheme.

The program uses the addresses generated by the compiler and further allocated by the Operating System,
the memory space correspond to the virtual memory address space of the program.
Note that each program has their own virtual memory address space.

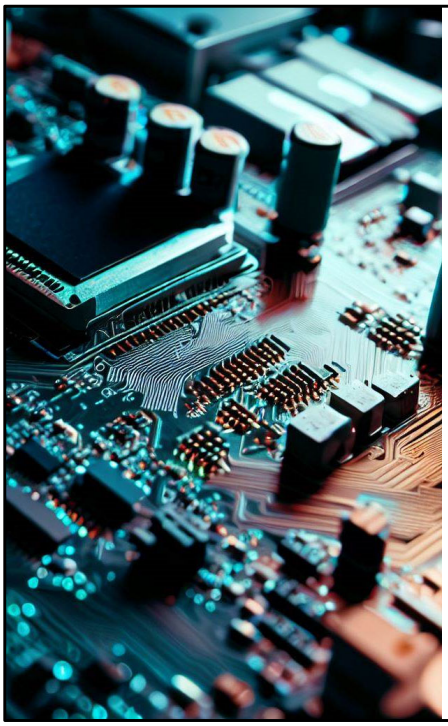
The actual program code and data are stored in the storage and system memory.
Specifically, as mentioned previously, the entire program code/data resides in the storage memory while the system memory stores the subset of the program that is needed during runtime.

As illustrated, the same piece of information, e.g. a test video clip, is stored in a different manner in the three entities.
In the program's virtual memory space, the test video could reside in contiguous piece of memory

In the System memory, it could be distributed across different frames in the system memory.
While in the storage memory, it may be allocated in random LBA
(Logical block addressing) blocks in the HDD.

Similar to the analogy in the previous slide, the program will still be able to function properly, as long as when it needs the test video, the corresponding data would be made available to the program.

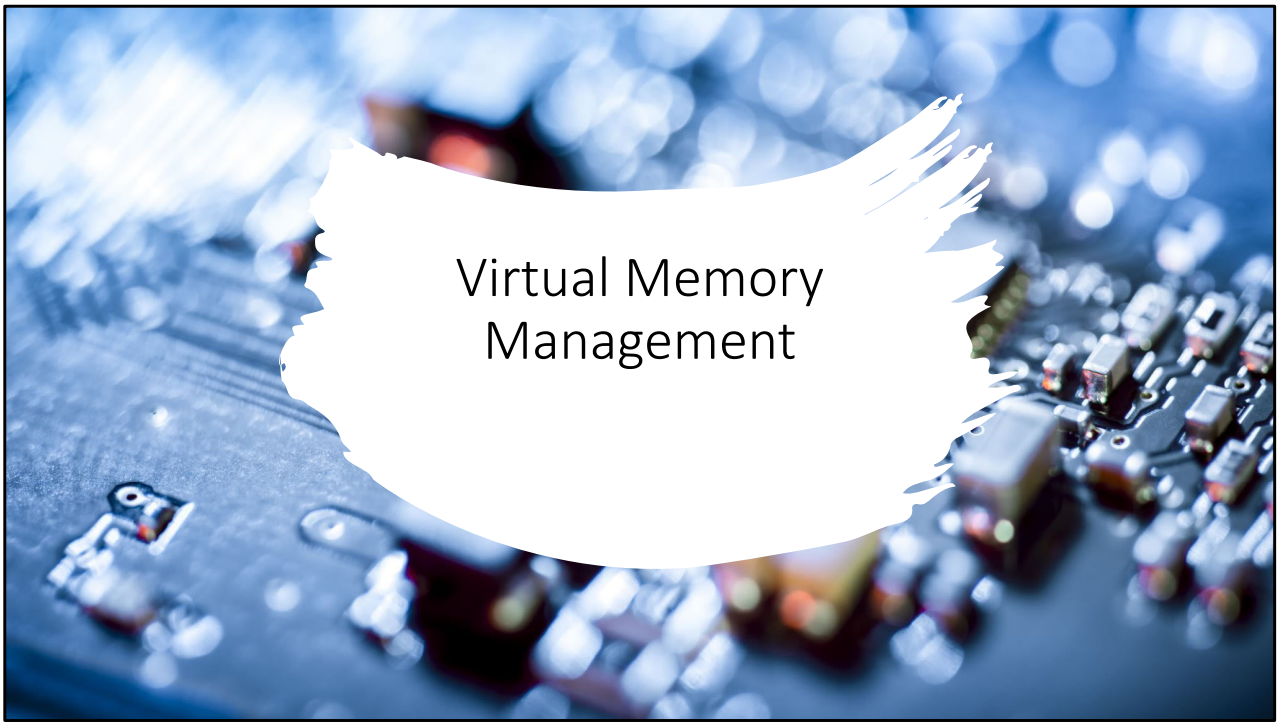
Since the test video is stored in different manner across the entities, that mean that some form of address translation have to occur in order for the program to retrieve the data correctly. The address translation is done by the OS. And we will go into details of one of the schemes employed.



Basic Concepts

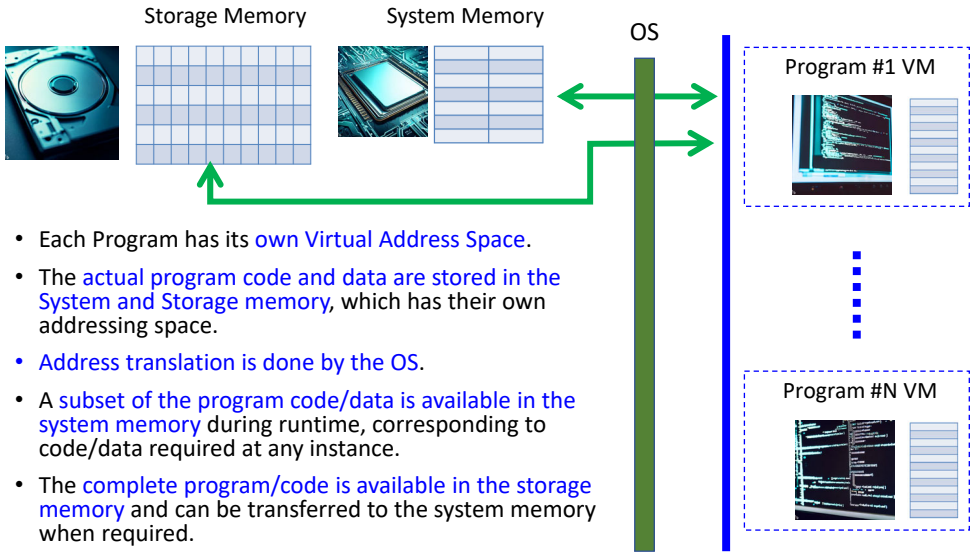
- In computing, address space is a range of discrete addresses corresponding to some physical or logical entity, e.g. program virtual memory, physical system memory, logical layout of a HDD etc.
- Every piece of data/code in a program is assigned an address by the compiler during the program compilation.
- When executing a program, the information it requires is obtained by issuing a request to the operating system (OS) using the addresses assigned by the compiler.
- The program doesn't need to know how the required information is organised in the system memory and rely on a middle man to do the corresponding address translation in order to fetch the correct information.
- In this case, the operating system is the middle man, translating the compiler generated address to the system memory address where the required information is stored.
- Program will still work as long as it gets the code/data it requested.

To recap, address space is a range of address corresponding to a physical or logical entity.
The address used by a program is generated by the compiler.
During program execution, OS will translate the compiler generated address to the system memory address where the information is physically stored.
The program doesn't need to know where the actual information is stored.
It will work properly as long as it gets the information it needs when it request for it.



- Next, we get into the Virtual Memory Management discussion.

Virtual Memory (VM) Management



- Each Program has its own Virtual Address Space.
- The actual program code and data are stored in the System and Storage memory, which has their own addressing space.
- Address translation is done by the OS.
- A subset of the program code/data is available in the system memory during runtime, corresponding to code/data required at any instance.
- The complete program/code is available in the storage memory and can be transferred to the system memory when required.

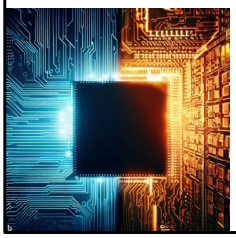
As mentioned in previous slides, each program has its own virtual address space. But the actual program code/data is stored in the system or storage memory, which has its own address space.

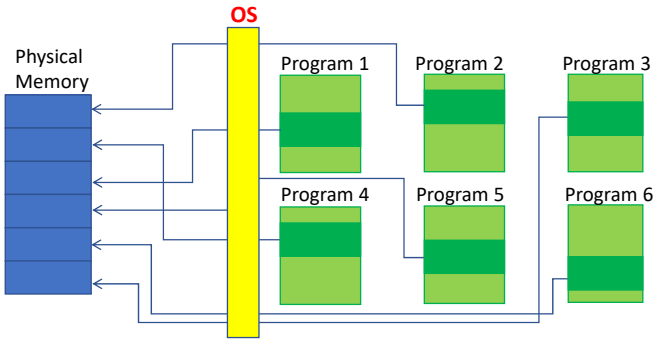
Therefore, address translation needs to be done in order for the program to retrieve the information it needs, and this translation is done by the OS.

Functionality wise, system memory stores the code/data required during any instance at runtime while the storage memory stores the entire program code/data.

Virtual vs Physical Memory Address Space

- The addresses used by the program is generated by the compiler and is known as the virtual address.
- The virtual address of the code/data is typically different from the Physical Memory Address that they will be resided.
- Address Translation is thus required and is done in hardware and/or software, managed by the Operating System (OS).
- Note that Physical, System and Main memory refers to the same memory for this course.

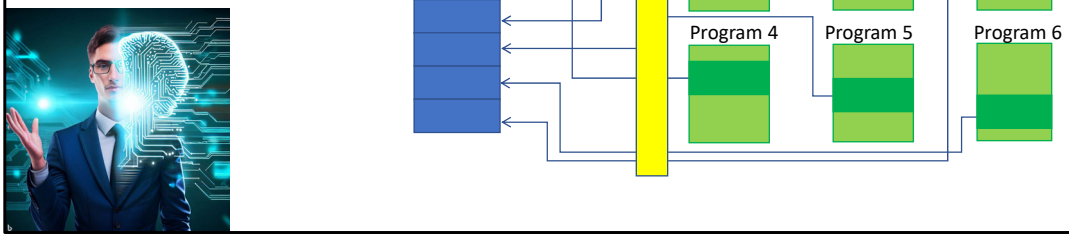




- Similar to the use of “Main Memory” in the Cache Chapter to refer to the System memory, in this chapter, System memory is sometimes referred to as the “Physical Memory”, basically to emphasize the actual runtime memory which the program code/data is stored in, this versus the Virtual Memory which is really a logical entity.
- The OS is responsible in managing the virtual memory, which includes allocating the subset of the program to the physical memory and performing the address translation from virtual to physical during program execution.
- The slide here shows the scenario where subset from multiple programs are allocated to the physical program simultaneously, with the OS handling the corresponding address translation.

Advantages of Virtual Memory

- OS is able to isolate each virtual memory space and prevent corruption between these spaces.
- Allow efficient and safe sharing among different programs within the shared physical memory.
- Allow one or more programs to run in the physical memory simultaneously even if the total size of all the programs is larger than the actual physical memory size.



Some of the advantages of having a virtual memory management system are:

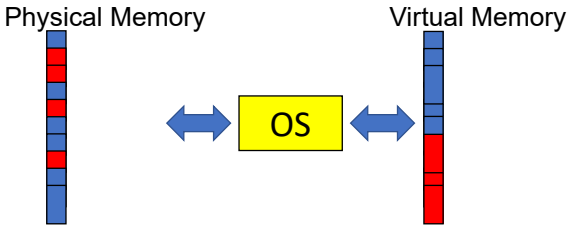
Virtual Memory space of each program can be isolated from each other to prevent intended or unintended corruption between programs.

This allows efficient sharing among different programs with the shared physical memory

Virtual Memory management allows multiple programs to run simultaneously in the physical memory even if the total size of all the programs are larger than the physical memory size. This is illustrated in the diagram below where 6 programs shared a physical memory whose size is smaller than the total of the 6 programs; e.g., Virtual memory uses both hardware and software to enable a computer to compensate for physical memory shortages, e.g., **temporarily transferring data from random access memory (RAM) to disk storage**.

Advantages of Virtual Memory

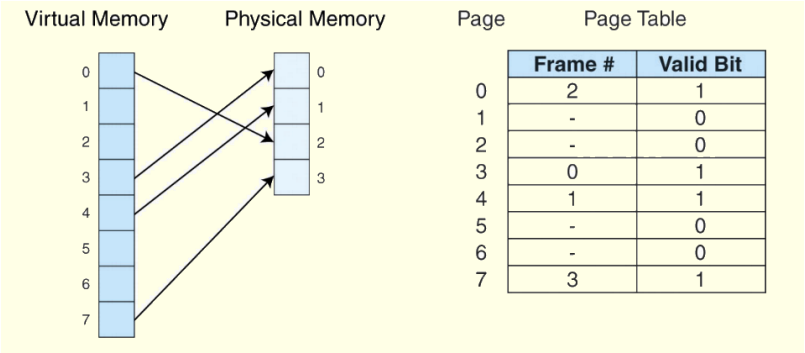
- OS is able to relocate virtual memory blocks to any physical memory blocks. This allows the virtual address space seen by the application to appear to be contiguous when it is actually spread across fragmented blocks in the physical memory.



- Another advantage of Virtual Memory Management is that the virtual address space seen by the program can appear to be contiguous when it is actually spread across different segments of physical memories.
- Having a contiguous piece of memory greatly simplify coding as the software does not need to do complicated boundary condition checking and management.

Address Mapping

- There are two common schemes used in the industry for address mapping
  - Paging. Memory space partitioned into Fixed sized blocks
  - Segmentation. Memory space partitioned into variable sized segments.
- For our course, we will only deal with paging with single level page table.



Page	Frame #	Valid Bit
0	2	1
1	-	0
2	-	0
3	0	1
4	1	1
5	-	0
6	-	0
7	3	1

Source: Linda Null

There are two common schemes used in the industry for address mapping.

Paging and Segmentation.

We will only cover Paging in the course.

For Paging, the virtual and physical memory space are divided into fixed size pages and frames respectively.
We use the name ‘Page’ for Virtual Memory and ‘Frame’ for Physical Memory.

The OS will map a virtual page to a particular physical frame and the translation information is store in a data structure known as a Page Table. More on this in later slides.

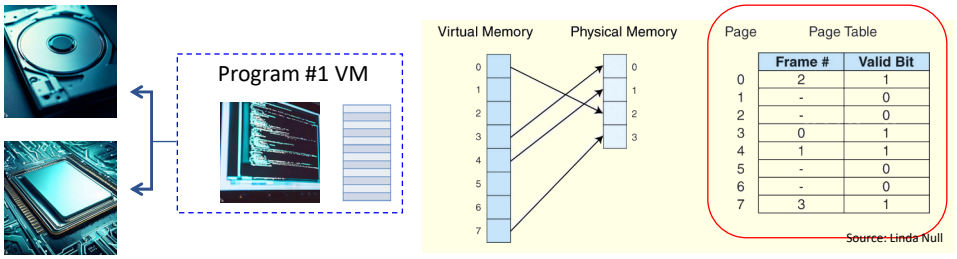
Our course will only cover paging with single page table, you will progress with multi-level page table in your Operating System Course.

More explanation:
Paging scheme divides virtual memory into equal size pages.

These pages are mapped to a frame in the physical memory
The mapping information is stored in the page table. E.g. virtual page 0 is mapped to physical frame 2 in the table in the slide.
So when the program thought it is running its code from Virtual Page 0 addresses, the processor is actually accessing Frame 2 addresses in the Physical memory.

Paging Method

- In a system that uses paging, the memory space is partitioned into fixed size blocks known as a Page/Frame.
- Information concerning the location of each page, whether on disk or in memory, is maintained in a data structure called a page table (shown below).
- In the Page Table below
  - Frame refers to the physical frame number in the main memory.
  - Page refers to the virtual page number used by program code.
  - Valid Bit (VB) indicates whether the Virtual Page is in the main memory (VB=1) or not (VB=0).



The size of a Virtual Page is always the same as the size of a Physical Frame in Paging Scheme of address mapping.
The OS will allocate a particular virtual page used by the program to be stored in a particular Frame in the physical memory during runtime.

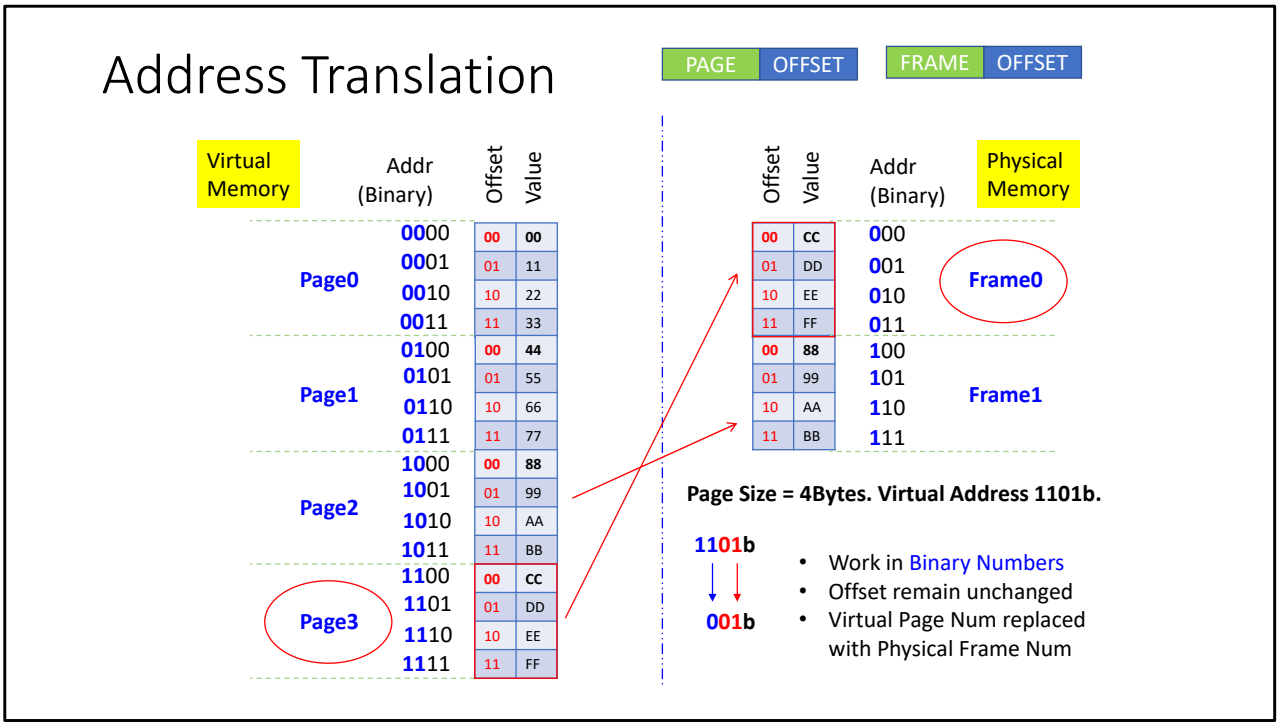
The mapping information is kept in the Page Table shown.

Within the Page table, you will see at least three parameters

Virtual Page column containing the virtual page number of the target virtual page.

Frame column containing the corresponding physical frame that the virtual page is mapped to.

And the Valid column showing whether the particular entry is valid or not.
If the valid bit is '1', this implies that corresponding mapping is valid and the information in that particular page is in the physical memory at that instance. E.g. in the page table shown, Virtual Page 0 information is resided in the Physical Frame 2 at that instance.



This slide illustrates the process of address translation from Virtual to Physical Memory Space via a work example.

The system shows here has the following attributes:
Virtual Memory Space of 16 bytes.
Physical Memory Space of 8 bytes.
Virtual Page size of 4 bytes, meaning the Physical Frame size is also 4 bytes.

First point to note is that when we map a virtual page to a physical frame, the relative position of the data in the page does not change.
You can see that when Page 3 is mapped to Frame 0, the offset of the 4 bytes of data in the Page does not change. E.g. offset of the data '0xEE' is 2 for decimal (or 10 in binary) in the Virtual Page3, the same data "0xEE" has an offset of 2 as well in the Physical Frame 0.

With that, let's start the address mapping process.
First, convert the address to binary format. This is an important step, never work in hexadecimal format, there is a high possibility that you will make careless mistakes.
Next, we need to partition the Virtual Address to two fields – Page and Offset.

This is done starting with the offset

Using the fact that page size is 4, then number of bits allocated to the Offset Field is 2. LOG2(4) = 2.

We can deduce the virtual page number from the upper address bits of the virtual address and use that to look up the page table to find any matching Physical Frame Number.

If the entry for the particular page number is valid, the Corresponding Frame Number is used to replace the upper address bits to give the Physical Memory Address the virtual address is mapped to.

Let's put some real values to illustrate the translation.

Starting with a virtual address of 1101 in binary.

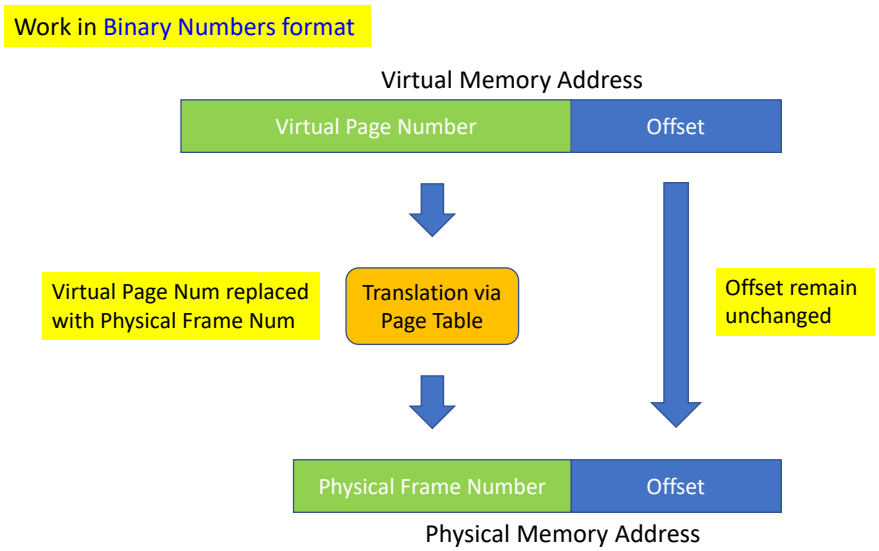
Offset field is 2 bits so the address 1101 is stored in virtual page 3, 11b are the upper bits.

Page 3 is mapped to Frame 0 from the diagram.

So to derive the physical memory address, we replace '11' in binary with '0'. This gives the Physical Memory address 001 in binary.

You can try out other Virtual Memory Addresses in this slide to reinforce your concept.

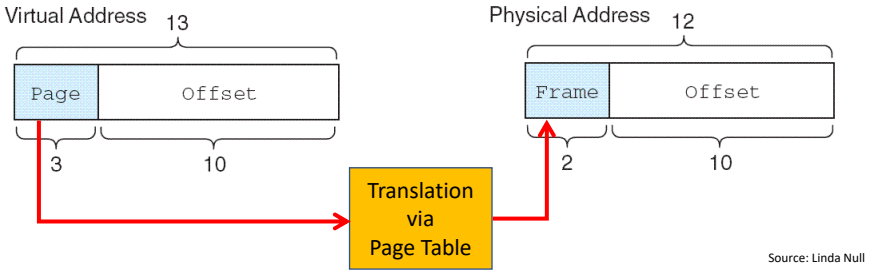
Address Translation



- A graphics illustration of the address translation process we discussed in previous slide.
- Convert the addresses to binary.
- Derive the number of bits in the offset field from the virtual page size.
- Partition the Virtual Memory Address to two fields: Virtual Page Number and Offset.
- The Offset remains unchanged during the translation.
- Use the Virtual Page Number to look up the Page Table for a valid mapping
- If a valid mapping is found, extract the corresponding Frame Number to replace the Virtual Page Number with the Physical Frame Number to have a Physical Memory Address.

Paging Example

- Consider a system with a virtual address space of 8K and a physical address space of 4K, and the system uses byte addressing.
- If page size is 1KByte, we have 8 virtual pages mapping to 4 physical frames.
- Note that Virtual Page Size is always equal to Physical Frame Size.
- A virtual address has 13 bits (8K = 2^{13}) with 10 for the offset, because the page size is 1024, and 3 bits for the page field.
- A physical memory address requires 12 bits, the first two bits for the page frame and the trailing 10 bits the offset.



Source: Linda Null

More work example.

Consider a system with
Virtual Address Space 8KByte
Physical Address Space 4Kbyte
Page size of 1KByte

From the attributes, we know that the system has 8 virtual pages and 4 physical frames.

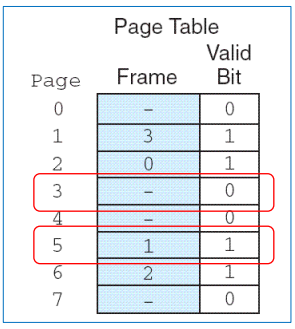
The page size is 1KByte, which is 2^10 bytes. That implies the OFFSET field is 10 bits.

Virtual address is 13 bits and Physical Address is 12bits.

So the translation process involves replacing the Upper 3 bits of the Virtual Address with the 2 bit Frame Number from the Page Table entries.

Paging Example

- Suppose we have the page table shown below.
- What happened when CPU access virtual address location
  - 0x1553
  - 0x0FA0



Page Table

Page	Frame	Valid Bit
0	–	0
1	3	1
2	0	1
3	–	0
4	–	0
5	1	1
6	2	1
7	–	0

Source: Linda Null

- 0x1553 = 1010101010011b
  - Data resides in Virtual Page 5
  - From Page Table, Virtual Page 5 is mapped to Physical Frame 1 and Valid bit is 1.
  - Physical Address = 010101010011b = **0x0553**
- 0x0FA0 = 0111110100000b
  - Data resides in Virtual Page 3
  - From Page Table, Valid bit is 0
  - Data not in Physical Memory
  - **Page Fault** (triggers the OS to load the required page from storage memory to the Physical Memory)

Putting in some values for the example in the previous slide.

0x1553 correspond to virtual page number 5.
Looking at the page table, we see that virtual page 5 has its valid bit set, meaning that it is mapped to some physical memory and from the same table, we see that the its frame #1.
Translation process involves replacing the upper 3 bits (101b) in Virtual Memory Address with 1.

For 0x0FA0, it corresponds to virtual page number 3.
When we look at the page table, we can see that its valid bit is 0. Meaning no physical memory is mapped to this virtual page.
This generates a page fault that triggers the OS to load the required page from storage memory to the Physical Memory.

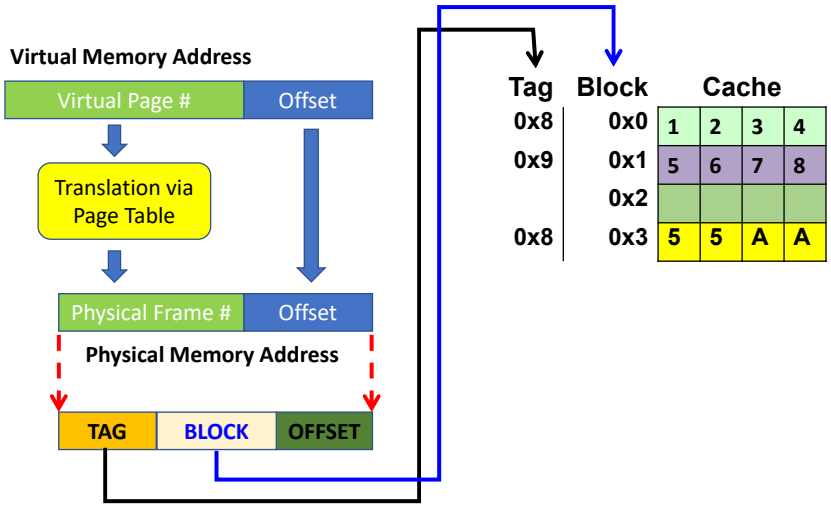
Paging Example

- In the previous slide, when accessing virtual address 0x1553 (which is in virtual page 5), the page table shows the valid bit is 1. From the page table, physical frame 1 will be used for subsequent address translation needed to access the actual data.
- When accessing virtual address 0xFA0 (virtual page 3), the page table shows that the valid bit is zero. If the valid bit is zero in the page table entry for the virtual address, this means that the page is not in main memory and must be fetched from Storage Memory.

- This slide summarises the discussion we did in the previous slide.
- You can pause the video for a while to review the points again.

Integrating Cache and Virtual Memory

- Assumption: Cache uses Physical Memory Address to perform Mapping.



In a typical computer system, especially one with an operating system, both cache and virtual memory management will be implemented.

Therefore, there is a need to look at integration of these two modules.

This slide shows one of the ways in which the processor may retrieve the information required when executing a program

With a software program, we would always start with the virtual memory address.

If we assume that the cache in this processor uses physical memory address to perform the indexing and mapping, then the following procedure will be carried out within this computer system.
First, the virtual memory address will be translated to its corresponding physical memory address using the methods we discussed in previous slides.
The CPU then anaylses if there is a cache hit or cache miss using the target's physical address.
If it is a cache Hit, then CPU will retrieve the information from the cache and proceed.
If it is a cache miss, then cache management module will proceed to perform the

transfer of data from System/Storage memory.

For reading after class if interested in:

Note that the sequence will be slightly different if the Cache is using Virtual Memory Address to do the indexing and mapping.

Starting with the virtual address used by the program again, the CPU will visit the cache first to check if it is a cache hit or cache miss.

If it is a cache hit, CPU will retrieve the data and proceed.

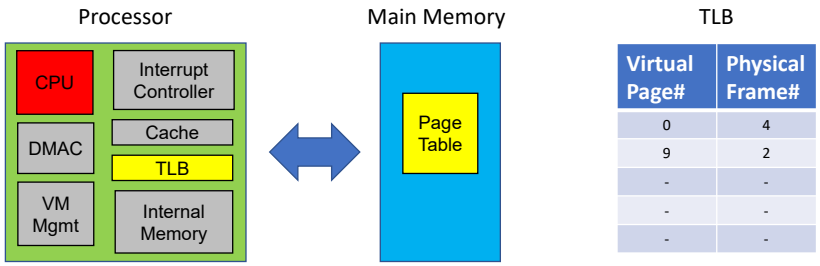
If it is a cache miss, that means the block containing the required data needs to be fetched from the Main Memory.

But the OS will have to translate the virtual memory address to physical memory address before the data can be fetched.

As seen, the sequence of access and translation is different but the concept behind each of the cache mapping and virtual memory translation is the same.

Translation Lookaside Buffer (TLB)

- Page Table is located in Main Memory so access is slower than internal memory access.
- To speed up the page translation process, a page table cache (TLB) is used to stored a list of most recently/frequently used address translation entries in the page table, as a subset of Page Table.
- TLB is implemented with fast memory such as SRAM and reside within the processor.
- Each TLB entry includes a Virtual Page number and its corresponding Frame Number.



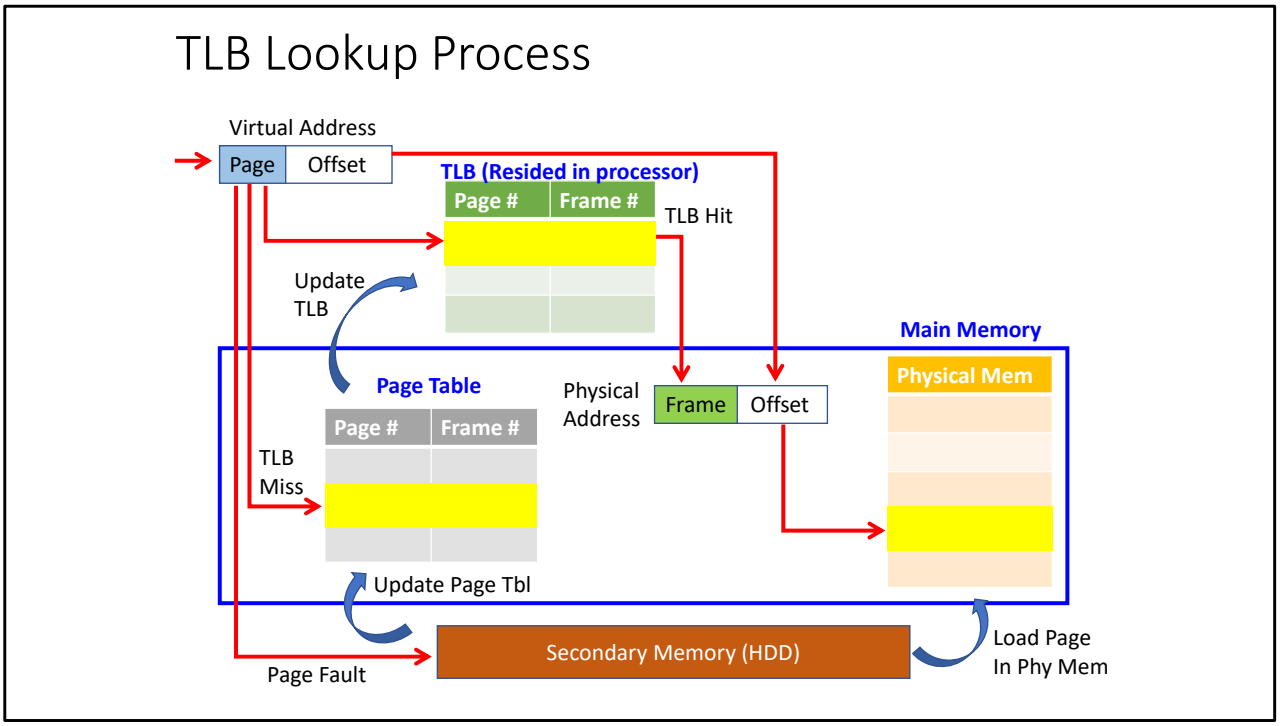
Due to its size, which could go up to a few MBs, Page table is located in the Main Memory, which is slower compared to the internal memory.

Since the process of translating Virtual Memory Address to Physical Memory Address happen very frequently, it makes sense to optimise this process.
One way is to use a fast internal memory, called a TLB (Translation Lookaside Buffer) to store frequently used page table information.
And get the OS to refer to the TLB first when they are looking for address translation information.
If the hit rate of the TLB is high, the overall speed of the address translation will in crease significantly.

TLB is implemented with fast memory like SRAM in the processor.
It stores a subset of the Page Table information.
Each TLB entry includes the Virtual Page Number and the corresponding Physical Frame it is mapped to.
All entries in the TLB are Valid entries.

So in some way, TLB function like a cache to the Page Table.

But the difference between a TLB and the Cache we discussed previously is that a TLB only stores address translation information i.e. which virtual page gets mapped to which physical frame. While a regular cache stores the actual code and data of a program.



The slide here illustrates the 3 ways that the virtual to physical memory address translation could be carried out.

Starting with the Virtual Memory Address.

1)The OS will check the TLB for a matching entry.
If the virtual page being access has an entry in the TLB, then it is a TLB hit.
The OS can extract the corresponding Physical Frame Number the virtual page is mapped to and proceed to derive the Physical Memory Address and access the data.

2)If the require entry is not in the TLB, then it is a TLB miss.
OS will proceed to check the Page Table resided in the Main Memory for the information.
If a valid entry if found in the Page Table, it is a Page Table Hit, the OS proceed to update the TLB, compute the physical address and access the data.

3)Under the worst case scenario, the entry in the Page Table is invalid, that means the required data is not in the Physical memory and the OS needs to transfer the data from the Storage memory to the Physical Memory and update the page table.

Whether the TLB gets updated or not (in both TLB Miss and Page Fault) will depend on the Virtual Memory Management design.

That is the end of the chapter on Virtual memory management.